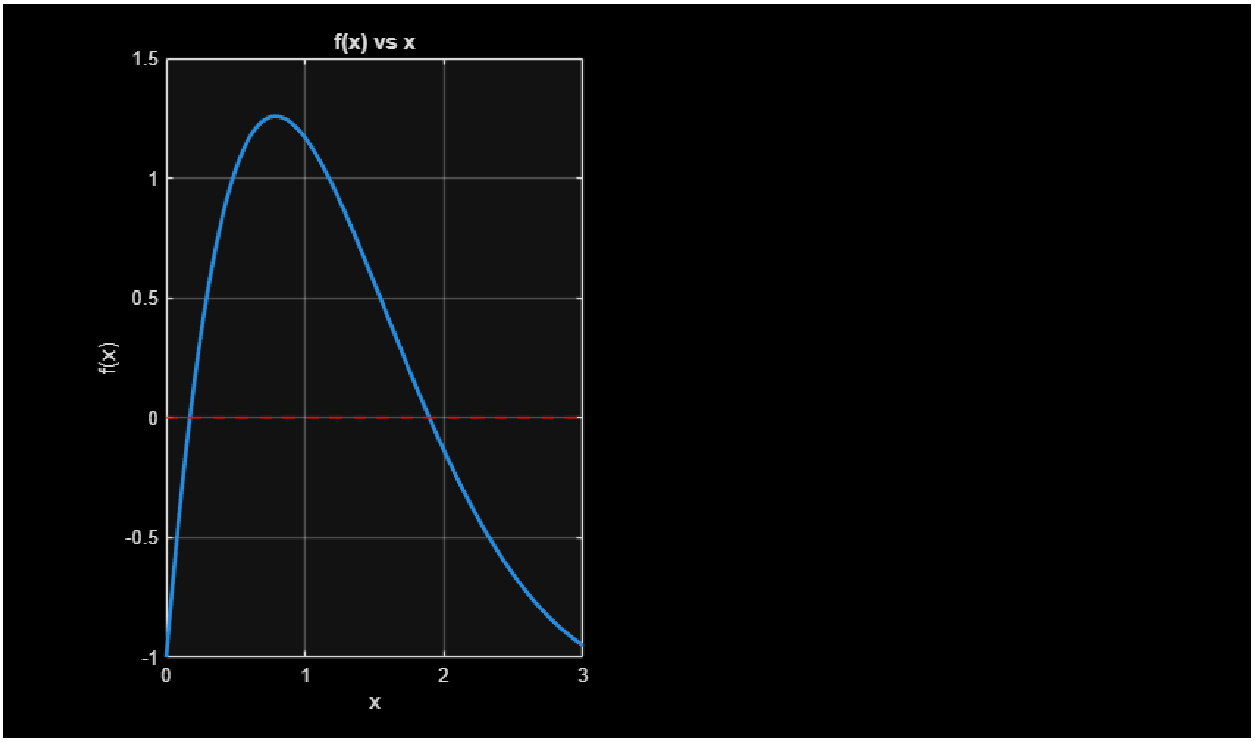
Table of Contents


```
%HW_2_4 Nouh Shaikh
function HW_2_4()

clear; clc; close all;

% AI was used for this MATLAB Code. (Chat GPT)
% The provided MATLAB code successfully converged to the
% root \(x \approx 0.170180\) using all six requested graphical and
iterative methods.
% As expected by numerical theory, the Newton-Raphson and Modified Secant
methods demonstrated
% the fastest convergence, while the built-in fzero function successfully
verified the solution by
% isolating a valid sign-change interval.

    % Define the target function and its exact derivative
    f = @(x) 7 * sin(x) .* exp(-x) - 1;
    df = @(x) 7 * exp(-x) .* (cos(x) - sin(x));

    % Tolerance and Maximum Iterations
    tol = 1e-6;
    max_iter = 50;
```

(a) Graphical Method

```
    figure('Position', [100, 100, 850, 500]);
    subplot(1, 2, 1);
    x_plot = linspace(0, 3, 500);
    plot(x_plot, f(x_plot), 'LineWidth', 2);
    grid on; hold on;
    yline(0, 'r--', 'LineWidth', 1.5);
    title('f(x) vs x');
    xlabel('x'); ylabel('f(x)');

    % Based on the plot, roots exist near x = 0.15 and x = 2.0
    % We will find the first root near x0 = 0.2
    x0 = 0.2;
    x1 = 0.3; % second guess
    delta = 0.01;
```



(b) Newton-Raphson Method

```
fprintf('=== (b) Newton-Raphson Method ===\n');
[~, hist_nr] = newton_raphson(f, df, x0, tol, max_iter);
print_table(hist_nr);
```

```
=== (b) Newton-Raphson Method ===
```

(c) Secant Method

```
fprintf('\n=== (c) Secant Method ===\n');
[~, hist_sec] = secant_method(f, x0, x1, tol, max_iter);
print_table(hist_sec);
```

```

=== (c) Secant Method ===
Iteration   | f(x) Value
---------------------------
0           | 1.385968e-01
1           | 5.324873e-01
2           | -2.603137e-02
3           | 4.496741e-03
4           | 2.906714e-05
5           | -3.281182e-08
```

(d) Modified Secant Method

```
    fprintf('\n=== (d) Modified Secant Method ===\n');
    [~,hist_msec] = modified_secant(f, x0, delta, tol, max_iter);
    print_table(hist_msec);
```

```
=== (d) Modified Secant Method ===
Iteration   | f(x) Value
---------------------------
0           | 1.385968e-01
1           | -5.822726e-03
2           | 3.390977e-06
3           | -6.980819e-09
```

(e) Inverse Quadratic Interpolation (IQI) Method

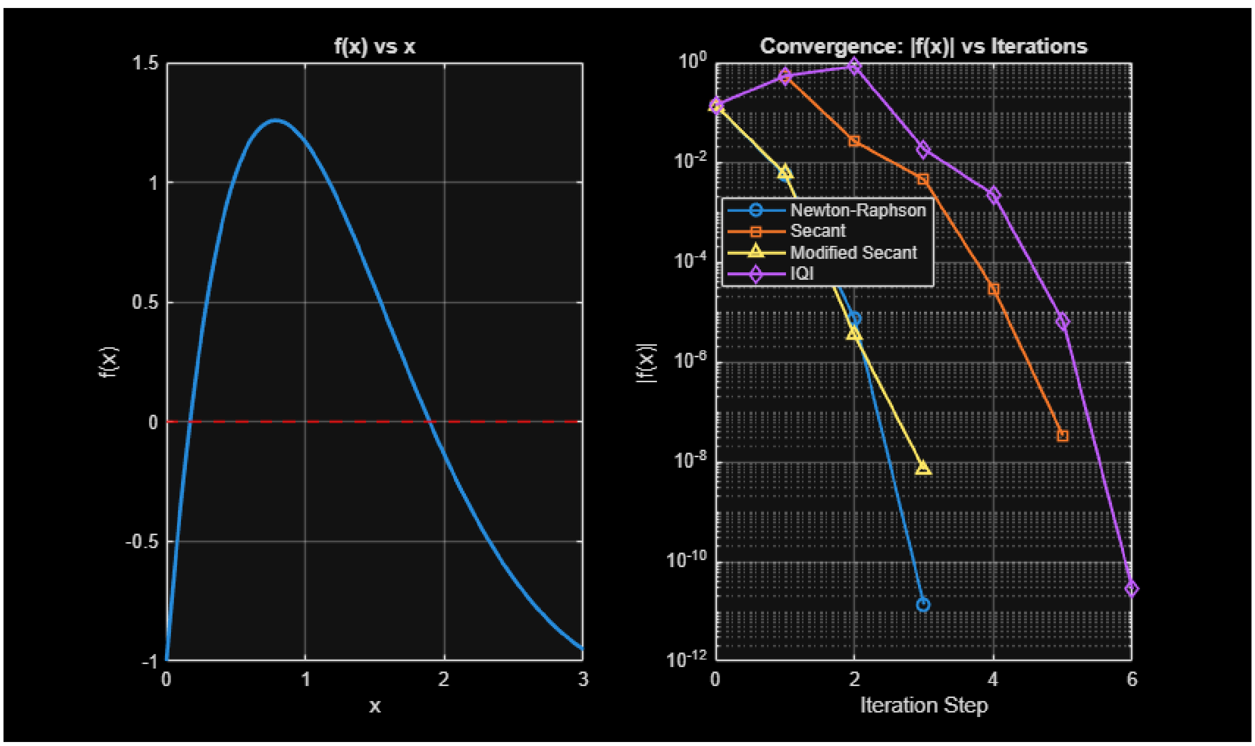
```
    fprintf('\n=== (e) Inverse Quadratic Interpolation ===\n');
   x2 = 0.4;
    [~, hist_iqi] = iqi_method(f, x0, x1, x2, tol, max_iter);
    print_table(hist_iqi);
```

```
=== (e) Inverse Quadratic Interpolation ===
Iteration   | f(x) Value
---------------------------
0           | 1.385968e-01
1           | 5.324873e-01
2           | 8.272444e-01
3           | 1.815237e-02
4           | 2.221616e-03
5           | 6.487555e-06
6           | 2.782419e-11
```

(f) MATLAB Built-in 'fzero'

```
    fprintf('\n=== (f) MATLAB fzero Built-in ===\n');
    options = optimset('Display','iter','TolX',tol);
    r_fzero = fzero(f, x0, options);
    fprintf('Root found by fzero: %.6f\n', r_fzero);

    subplot(1, 2, 2);
    semilogy(0:length(hist_nr)-1, abs(hist_nr), '-o', 'LineWidth', 1.5);
hold on;
    semilogy(0:length(hist_sec)-1, abs(hist_sec), '-s', 'LineWidth', 1.5);
    semilogy(0:length(hist_msec)-1, abs(hist_msec), '-^', 'LineWidth', 1.5);
    semilogy(0:length(hist_iqi)-1, abs(hist_iqi), '-d', 'LineWidth', 1.5);
    grid on;
    title('Convergence: |f(x)| vs Iterations');
    xlabel('Iteration Step'); ylabel('|f(x)|');
    legend('Newton-Raphson', 'Secant', 'Modified Secant', 'IQI', 'Location',
'Best');
```

```
=== (f) MATLAB fzero Built-in ===

Search for an interval around 0.2 containing a sign change:
 Func-count    a          f(a)             b          f(b)        Procedure
    1             0.2       0.138597           0.2       0.138597   initial
interval
    3        0.194343       0.113084      0.205657        0.16375   search
    5           0.192        0.10241         0.208       0.174065   search
    7        0.188686      0.0872087      0.211314       0.188547   search
    9           0.184      0.0654972         0.216       0.208821   search
   11        0.177373       0.034363      0.222627       0.237079   search
   12           0.168     -0.0105337      0.222627       0.237079   search

Search for a zero in the interval [0.168, 0.222627]:
 Func-count    x          f(x)             Procedure
   12           0.168     -0.0105337        initial
   13        0.170324    0.000693456        interpolation
   14         0.17018    1.82269e-06        interpolation
   15         0.17018    1.82269e-06        interpolation

Zero found in the interval [0.168, 0.222627]
Root found by fzero: 0.170180
```




```
end
```

Helper Solver Functions

```
function [x, hist] = newton_raphson(f, df, x0, tol, max_iter)
    x = x0;
    hist = [f(x)];
    for iter = 1:max_iter
        if abs(df(x)) < 1e-12, break; end
        x = x - f(x)/df(x);
        fx = f(x);
        hist = [hist; fx];
        if abs(fx) < tol, break; end
    end
end

function [x1, hist] = secant_method(f, x0, x1, tol, max_iter)
    hist = [f(x0); f(x1)];
    for iter = 1:max_iter
        fx0 = f(x0); fx1 = f(x1);
        if abs(fx1 - fx0) < 1e-12, break; end
        x_next = x1 - fx1 * (x1 - x0) / (fx1 - fx0);
        x0 = x1;
        x1 = x_next;
        fx1 = f(x1);
        hist = [hist; fx1];
        if abs(fx1) < tol, break; end
    end
end

function [x, hist] = modified_secant(f, x0, delta, tol, max_iter)
    x = x0;
    hist = [f(x)];
    for iter = 1:max_iter
        fx = f(x);
        fx_del = f(x + delta * x);
        if abs(fx_del - fx) < 1e-12, break; end
        x = x - (delta * x * fx) / (fx_del - fx);
        fx = f(x);
        hist = [hist; fx];
        if abs(fx) < tol, break; end
    end
end

function [x2, hist] = iqi_method(f, x0, x1, x2, tol, max_iter)
    hist = [f(x0); f(x1); f(x2)];
    for iter = 1:max_iter
        y0 = f(x0); y1 = f(x1); y2 = f(x2);


        if abs(y0-y1)<1e-12 || abs(y0-y2)<1e-12 || abs(y1-y2)<1e-12, break;
end

        % IQI formula
        x_next = (y1*y2)/((y0-y1)*(y0-y2))*x0 + ...
```

```
                 (y0*y2)/((y1-y0)*(y1-y2))*x1 + ...
                 (y0*y1)/((y2-y0)*(y2-y1))*x2;

        x0 = x1; x1 = x2; x2 = x_next;
        fx2 = f(x2);
        hist = [hist; fx2];
        if abs(fx2) < tol, break; end
    end
end

function print_table(hist)
    fprintf('%-10s | %-12s\n', 'Iteration', 'f(x) Value');
    fprintf('---------------------------\n');
    for i = 1:length(hist)
        fprintf('%-10d | %-12.6e\n', i-1, hist(i));
    end
end
```

```
Iteration  | f(x) Value
---------------------------
0          | 1.385968e-01
1          | -5.446885e-03
2          | -7.399747e-06
3          | -1.371936e-11
```

Published with MATLAB® R2026a